3. FAST API BUILDING ROBUST APIs

3.1 Setting Up your fast api inventory project

In this section, we begin building a **production-style FastAPI application** by setting up a modular project structure, configuring the environment, and initializing the

Project Overview

The goal is to build an **inventory management system** that tracks:

- Products
- Storage locations
- Inventory levels

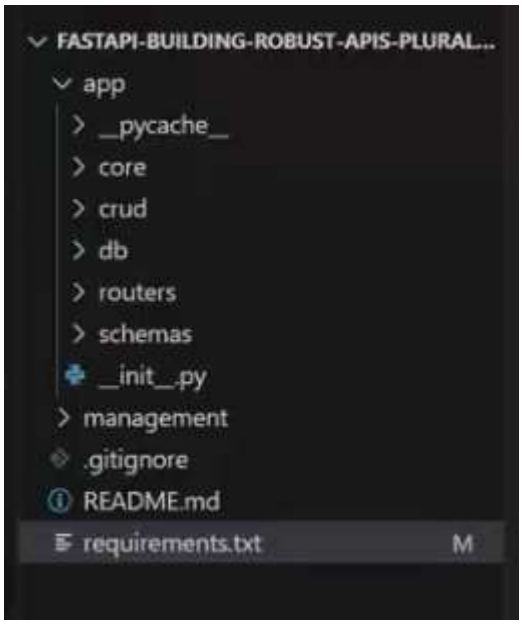
This project follows a **modular architecture**, making it scalable and maintainable.

Project Structure

FastAPI does not enforce a strict project structure. However, for medium to large applications, a modular approach is highly recommended.

Recommended Directory Layout

- **crud/**
  Contains database operation logic (Create, Read, Update, Delete).
- **db/**
  Defines database models and connection setup.
- **routers/**
  Contains API endpoints grouped by functionality.
- **schemas/**
  Defines data validation and serialization models (using Pydantic).
- **management/**
  Includes utility scripts such as inventory_tools.py.



- Fastapi has no official project structure – this is just an approach that works well for medium to large application
-

Why This Structure Works

- Separation of concerns
- Easier debugging and testing
- Scales well with team size and feature growth

Environment Configuration (.env)

Environment variables are used to manage configuration such as:

- Database connection strings
- Secret keys
- Runtime settings

Best Practice

- Use a .env file during development
- Avoid using .env directly in production (use secure configuration methods)

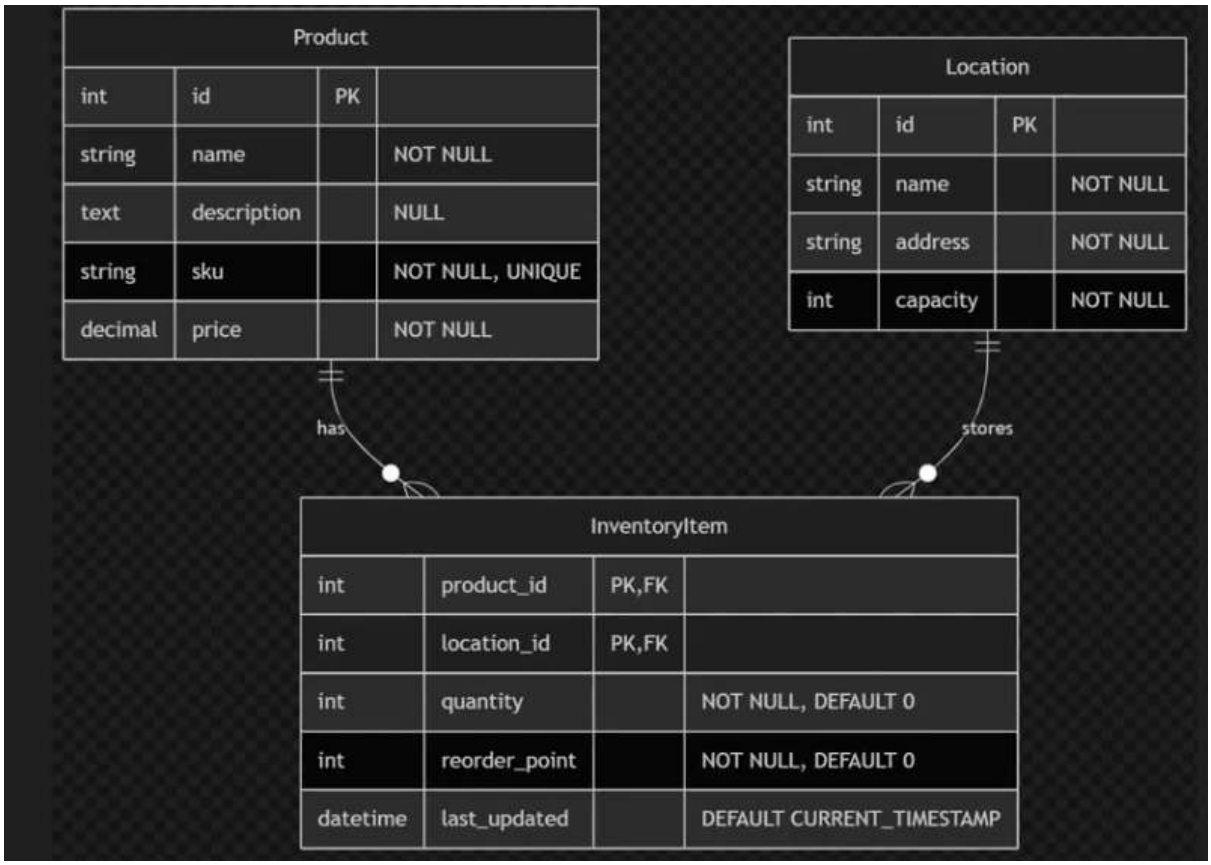
Database Setup

The application uses **SQLite** for simplicity during development.

Core Tables

1. **Product**
    - Stores product-related information
2. **Location**
    - Represents warehouses or storage locations
3. **Inventory**
    - Tracks quantity of each product at specific locations

These tables form the foundation of the inventory system.



Installing Dependencies

Steps:

- **Create Virtual Environment :** Isolate project dependencies

```
python -m venv venv
```

- **Activate Environment :** Ensure all installations are scoped locally

```
venv\Scripts\activate
```

- **Install Dependencies :** Define all required packages in requirements.txt , Install them in one step

```
pip install -r requirements.txt
```

- **Running Utility Scripts**

  Inside the management/ directory:

  - **inventory_tools.py**
    - Used for initial setup tasks (e.g., database initialization)

  Run this script to prepare your environment before starting the API.

```
management/inventory_tool.py
```

- **Creating the FastAPI Application:**
  - Step 1: Create Entry Point
    - Create a main.py file as the application entry point.Importing fastapi from fastapi
  - Step 2: Initialize FastAPI
    - Import FastAPI
    - Create an application instance

```
from fastapi import FastAPI
# Create FastAPI app
app = FastAPI(
    title="Inventory Management API",
    description="API for managing inventory across multiple locations",
    version="1.0.0"
)
```

  - Step 3: Run the Application
    - Use uvicorn inside the __main__ block
    - This starts the FastAPI server

```
# For development, we'll run the app with Uvicorn
if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=8000, reload=True)
```

  - **Step 4: Add Health Check Endpoint**

- Create a simple endpoint:
- Route: /health
- Purpose: Verify that the application is running
- This is a standard practice in production systems for monitoring and uptime checks.

```
@app.get("/health", tags=["Health"])
async def health_check():
    """Health check endpoint to verify API is running"""
    return {
        "status": "online",
        "message": "Inventory Management API is running"
    }
```

- **API Documentation**
  - FastAPI automatically generates interactive API documentation:
  - **Swagger UI** → Available at /docs
  - **ReDoc** → Available at /redoc
  - **Benefits**
  - Test endpoints directly from the browser
  - Understand request/response formats
  - Useful for both developers and stakeholders

3.2 Building Data Models with Pydantic Validation

In this section, we extend our FastAPI inventory system by:

- Creating modular API endpoints
- Connecting them to the database
- Introducing **Pydantic** for data validation and schema design

This forms the backbone of a clean, scalable backend architecture.

- **Creating Modular API Routers**

To maintain scalability, FastAPI applications should organize endpoints into separate modules.

- **Create Router Directory**
  - Create a routers/ directory
  - Add files:
    - products.py
    - locations.py
    - inventory.py

  - Implementing Product Endpoints (products.py)
- **Step 1: Initialize Router**
  - Import APIRouter from FastAPI
  - Create a router instance

    Purpose:
  - Allows grouping related endpoints
  - Enables modular API design

```
from fastapi import APIRouter
```

- **Step 2: Configure Router**
  - Add:
    - prefix="/products" → All routes start with /products
    - tags=["Products"] → Organizes Swagger documentation

```
router = APIRouter(
    prefix="/products",
    tags=["Products"]
)
```

- **Step 3: Create Basic Endpoint**
  - Define a GET / endpoint:
    - Function: list_products
    - Returns a placeholder list initially

    This acts as a starting point before integrating the database.

```python
@router.get("/")
def list_products():
    """
    Retrieve all products
    """
    # For now, we'll just return an empty list
    # We'll connect to the database in the next step
    return []
```

- **Step 4: Register Router**

  Inside routers/__init__.py:

  - Import router:
    - products_router
  - Add it to a list:
    - routers = [products_router]

```python
from app.routers.products import router as products_router
routers = [
    products_router
]
```

- **Step 5: Attach Routers to Main App**

  In main.py:

  - Import routers list
  - Loop through routers:
    - app.include_router(router)

  Why this pattern?

  - Keeps the main file clean
  - Makes scaling to multiple modules easier

```python
from app.routers import routers
```

```python
for router in routers:
    app.include_router(router)
```

- Step 6: Add Database Integration

- Imports
- Session from SQLAlchemy
- get_db dependency
- product_repository (CRUD layer)

```
from sqlalchemy.orm import Session
from app.db.session import get_db
from app.crud.product import product_repository
```

- **Step 7: Dependency Injection**
  - Use Depends(get_db)
  - Automatically provides a database session

    Important Behavior:
  - Session is created per request
  - Closed safely using try...finally

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

- **Step 8: Implement Product Endpoints**

  Get All Products

  - Fetch data using repository layer
  - Return structured response

  Create Product (POST)

  - Accept validated input
  - Insert into database via repository

```
class ProductRepository:
    def get(self, db: Session, product_id: int) -> Optional[Product]:
        """Get a product by ID"""
        return db.query(Product).filter(Product.id == product_id).first()
```

```python
def get_by_sku(self, db: Session, sku: str) -> Optional[Product]:
    """Get a product by SKU"""
    return db.query(Product).filter(Product.sku == sku).first()

def get_all(self, db: Session, skip: int = 0, limit: int = 100) -> List[
    """Get all products with pagination"""
    return db.query(Product).offset(skip).limit(limit).all()
```

- Step 9: define database in your product.py file

```python
from fastapi import APIRouter,Depe
```

  - import ... Dependencies: The highlighted line appears to be importing APIRouter and initializing a Depends function for dependency injection.
- Step 10: creating a new endpoint that returns the details of all the products

```python
@router.get("/")
def list_products(db: Session = Depends(get_db)):
    """
    Retrieve all products
    """
    products = product_repository.get_all(db)
    return products
```

  Code breakdown:
  - The @router.get("/") decorator maps HTTP GET requests to the root path of the router.
  - The list_products function takes a database session db as a dependency.
  - It calls a repository method to fetch all products from the database and returns them.

- **Implementing Location Endpoints (locations.py)**
- Create router with:

  - prefix="/locations"
  - tags=["Locations"]

```
router = APIRouter(
    prefix="/locations",
    tags=["Locations"]
)
```

Code Breakdown

- **prefix="/locations"**: This automatically prepends the path /locations to any routes defined within this specific router.
- **tags=["Locations"]**: This organizes the endpoints under a specific category in the automatically generated OpenAPI documentation (Swagger UI).

- Define endpoints:
  - Retrieve all locations

```
@router.get("/")
def list_locations(db: Session = Depends(get_db)):
    """
    Retrieve all locations
    """
    locations = location_repository.get_all(db)
    return locations
```

- Register router in __init__.py

```
from app.routers.products import router as products_router
from app.routers.locations import router as locations_router
routers = [
    products_router,
    locations_router
]
```

Code Breakdown

- **Importing Routers**: The code imports router objects from distinct modules (app.routers.products and app.routers.locations) and aliases them for clarity.
- **Router List**: These imported routers are collected into a list named routers.
- **Modularization**: This structure allows developers to separate routes into different files or modules, improving code maintainability and scalability for larger applications.

- **Why Separate Files?**
  - Improves readability
  - Keeps domain logic isolated
  - Enables team collaboration

- **Implementing Inventory Endpoints (inventory.py)**
  The inventory module connects products and locations.
  - Step 1: Create Router
    - Define router with appropriate prefix (e.g., /inventory)

```
router = APIRouter(
    prefix="/inventory",
    tags=["Inventory"]
)
```

  - Step 2: Create Endpoints
    - Get Inventory by Product
      - Accept product ID as path parameter
      - Return inventory data
    - Get Inventory by Location
      - Accept location ID
      - Return stored items

```
@router.get("/by-product/{product_id}")
def get_inventory_by_product(product_id: int, db: Session = Depends(get_db))

    Retrieve all inventory items for a specific product
    """
    inventory_items = inventory_repository.get_by_product(db, product_id)
    return inventory_items
```

```
@router.get("/by-location/{location_id}")
def get_inventory_by_location(location_id: int, db: Session = Depends(get_db
    """
    Retrieve all inventory items at a specific location
    """
    inventory_items = inventory_repository.get_by_location(db, location_id)
    return inventory_items
```

  - Step 3: Exposing the new router in your init file

```python
from app.routers.products import router as products_router
from app.routers.locations import router as locations_router
from app.routers.inventory import router as inventory_router
routers = [
    products_router,
    locations_router,
    inventory_router
]
```

- Step 4: Accessing items and storing the result values as a dictionary

```python
(variable) result: list ry_repository.get_by_product(db, product_id)
result = []
for inventory_item, location in inventory_items:
    result.append({
        "quantity": inventory_item.quantity,
        "reorder_point": inventory_item.reorder_point,
        "location_name": location.name,
        "location_id": location.id,
        "product_id": inventory_item.product_id,
        "in_stock": inventory_item.quantity > 0,
        "needs_reorder": inventory_item.quantity < inventory_item.reorde
    })
return result
```

- Step 5: Handle Query Parameters
  - Example: skip
    - Used for pagination
    - Helps limit large responses

```python
def list_products(
    search: str = None,
    skip: int = 0,
    limit: int = 100,
    db: Session = Depends(get_db)
):
    """
```

```
    """
    Retrieve products with optional search and pagination
    """
    if search:                                  (parameter) db: Any
        return product_repository.search(db, search, skip=skip, limit=limit)
    else:
        return product_repository.get_all(db, skip=skip, limit=limit)
```

- Step 4: Process Data
  - Convert database results into dictionaries
  - Return structured API responses

Introduction to Pydantic

What is Pydantic?

Pydantic is a data validation library that:

- Uses Python type hints
- Automatically validates incoming data
- Converts types where necessary
- Provides clear error messages

Creating Data Schemas

- Step 1: Define Schema File
  - Create a file in schemas/ (e.g., product.py)


```
    """
    Retrieve products with optional search and pagination
    """
    if search:                                  (parameter) db: Any
        return product_repository.search(db, search, skip=skip, limit=limit)
    else:
        return product_repository.get_all(db, skip=skip, limit=limit)
```


- Step 2: Create Product Schema
  Example structure:
  - Class: ProductCreate
  - Fields:
    - name → string
    - description → string

        - sku → string
        - price → float
    - Step 3: Use BaseModel
        - Inherit from BaseModel
        - Enables validation and serialization

```
from pydantic import BaseModel
class ProductCreate():
    """Schema for creating a new product"""
    name: str
    description: str
    sku: str
    price: float
```

- Creating POST Endpoint with Validation Inside products.py:
    - Define POST / endpoint
        - **Key Components**
        - **Request Body**
        - Uses ProductCreate schema
        - **Dependency Injection**
        - Depends(get_db) provides database session
        - **Business Logic**
        - Calls product_repository.create()

```
from pydantic import BaseModel,Field
from typing import Optional
class ProductCreate(BaseModel):
    """Schema for creating a new product"""
    name: str= Field(..., min_length=1, max_length=100),
    description: Optional[str] = None
    sku: str = Field(..., min_length=5, max_length=9)
    price: float = Field(..., gt=0)
```

Why This Design Matters

- Keeps validation separate from logic
- Keeps route handlers clean
- Encourages layered architecture

Field-Level Validation

Pydantic allows validation at field level using validators.

Example Use Cases

- Price must be positive
- SKU format validation
- String length constraints

```
@field_validator('sku')
def validate_sku(cls, v):
    """Validate SKU format (e.g., TECH-001)"""
    if not re.match(r'^[A-Z]+-\d+$', v):
        raise ValueError('SKU must be in format CATEGORY-NUMBER (e.g.,
    return v.upper()  # Normalize to uppercase
```

```
@field_validator('name')
def validate_name(cls, v):
    """Validate name doesn't contain problematic characters"""
    forbidden_chars = ['@', '#', '$', '%', '&']
    for char in forbidden_chars:
      if char in v:
       raise ValueError(f"Name cannot contain special character '{char}
    return v
```

```
@field_validator('price')
def validate_price(cls, v):
    """Additional price validation"""
    # Round to 2 decimal places
    v = round(v, 2)

    # E (parameter) v: Any nreasonably high
    if v > 10000:
        raise ValueError('Price cannot exceed $10,000')
    return v
```

- **Demo: Model-wise Validation**

Sometimes validation requires multiple fields.

Using model_validator

This allows validation across the entire object.

Modes of Validation

1. **Before Mode**
   - Runs before type conversion
   - Used for preprocessing raw input
2. **After Mode**
   - Runs after field validation
   - Used for cross-field validation

Applying Validation in Inventory

Step-by-step implementation:

- Create a **PUT endpoint** in inventory.py

```python
@router.put("/")                    (import) InventoryUpdate: Any
def update_inventory(update: InventoryUpdate, db: Session = Depends(get_db))
    """
    Update inventory stock levels (add or remove stock)
    """

    result = inventory_repository.update_stock(
        db,
        product_id=update.product_id,
        location_id=update.location_id,
        quantity_change=update.quantity_change,
```

```python
        reorder_point=update.reorder_point
    )
    return result
```

- Define an inventory schema
- Add field validators

```python
class InventoryUpdate(BaseModel):
    """Schema for updating inventory stock levels"""
    product_id: int = Field(..., gt=0)
    location_id: int = Field(..., gt=0)
    quantity_change: int
    reorder_point: Optional[int] = Field(None, ge=0)
    reason: Optional[str] = None

    @field_validator('quantity_change')
    def validate_quantity_change(cls, v):
        """Validate quantity change is reasonable"""
        # Limit single transaction size to prevent errors
        if abs(v) > 100000:
            raise ValueError('Quantity change cannot exceed 1000 units in a
        return v
```

- Extend schema with model-level validation
- Apply logic to validate full request

```python
    @model_validator(mode='after')
    def validate_inventory_operations(self):
        """Validate business rules across multiple fields"""
        # For significant stock reductions, require a reason
        if self.quantity_change and self.quantity_change < -50 and not self.
            raise ValueError('Stock reductions of more than 50 units require

        # For extremely large changes in either direction, require detailed
        if self.quantity_change and abs(self.quantity_change) > 200 and (not
            raise ValueError('Changes of more than 200 units require a detai
```

3.3 Exception handling strategies

In real-world applications, failures are inevitable. A well-designed API must handle errors gracefully and provide **clear, consistent, and secure responses**.

This section explores exception handling in FastAPI, including:

- Route-level error handling
- Built-in exceptions
- Custom business exceptions
- Global exception handling

Why Exception Handling Matters

APIs must handle scenarios such as:

- Requested resources not existing
- Business rule violations (e.g., insufficient stock)
- Database failures
- Unexpected system errors

Without proper handling, FastAPI returns a generic:

500 Internal Server Error

This is not helpful for users and may expose internal issues.

Route-Level Error Handling

Example Scenario

Fetching a product by ID:

What happens if the product does not exist?

```python
@router.get("/{product_id}")
def get_product(product_id: int, db: Session = Depends(get_db)):
    """
    Retrieve a product by ID without any error handling
    """
    product = product_repository.get(db, product_id)

    # Simply return the product, even if it's None
    return product
```

Approach: JSON Response

Initially, we can manually return a JSON response.

Implementation Flow

- importing json:
  ```python
  from fastapi.responses import JSONResponse
  ```
- Accept product_id as input
- Query the database
- Check if the product exists \
- Return:
  - Product → if found
  - Error response → if not found

```python
@router.get("/{product_id}")
def get_product(product_id: int, db: Session = Depends(get_db)):
    """
    Retrieve a product by ID without any error handling
    """
    product = product_repository.get(db, product_id)
    if not product:
        return JSONResponse(
            status_code=status.HTTP_404_NOT_FOUND,
            content={
                "error": True,
                "message": "Product not found",
                "details": {"product_id": product_id}
            }
        )

    # Simply return the product, even if it's None
    return product
```

Using FastAPI's Built-in HTTPException

FastAPI provides a built-in exception mechanism:

- **HTTPException**

```python
def get_product(product_id: int, db: Session = Depends(get_db)):
    """
    Retrieve a product by ID without any error handling
    """
    product = product_repository.get(db, product_id)
    if not product:
        # Use HTTPException to set status code and response
        raise HTTPException(
            status_code=status.HTTP_404_NOT_FOUND,
            detail={
                "message": "Product not found",
                "details": {"product_id": product_id}
            }
        )
    return product
```

Improved Approach

Instead of manually returning JSON responses, we:

- Raise exceptions
- Let FastAPI handle formatting

Helper Function Pattern

To standardize error handling:

- Create a helper function (e.g., not_found())
- This function raises an HTTPException

```python
@router.get("/{product_id}")
def get_product(product_id: int, db: Session = Depends(get_db)):
    """
    Retrieve a product by ID without any error handling
    """
    product = product_repository.get(db, product_id)
    if not product:
        not_found("Product", product_id)
    return product
```

Benefits

- Cleaner route handlers
- Reusable logic
- Consistent error responses

Moving to Custom Exceptions

As applications grow, simple HTTP exceptions are not enough.

We need:

- Structured error handling

Custom Exception Design

Step-by-step implementation:

- Step 1: Create Exception Module
  - Create exceptions.py inside the core/ directory
  - Create exceptions.py file for creating base exception to catch globally
- Step 2: Define Base Exception
  - Create a base class (e.g., InventoryError)
  - All custom exceptions inherit from this

```python
class InventoryError(Exception):
    """Base exception class for inventory operations"""
    status_code = 500  # Default status code
                                        (parameter) details: dict
    def __init__(self, message: str, details: dict = None):
        self.message = message
        self.details = details or {}
        super().__init__(self.message)
```

- Step 3: Create Specific Exceptions

  RecordNotFoundError

  - Raised when a record does not exist
  - Includes:
    - Record type
    - Record ID
  - Returns:
    - Status code: 404

```python
class RecordNotFoundError(InventoryError):
    """Raised when a requested record doesn't exist"""
    status_code = 404

    def __init__(self, record_type: str, record_id: int):
        message = f"{record_type} with id {record_id} not found"
        details = {
            "record_type": record_type,
            "record_id": record_id
        }
        super().__init__(message, details)
```

- InsufficientStockError
  - Raised when stock goes below allowed level
  - Prevents invalid inventory operations

```python
class InsufficientStockError(InventoryError):
    """Raised when there isn't enough stock for an operation"""
    status_code = 400

    def __init__(self, product_id: int, requested: int, available: int):
        message = f"Insufficient stock for product {product_id}"
        details = {
            "product_id": product_id,
            "requested": requested,
            "available": available
        }
        super().__init__(message, details)
```

- **Integrating Exceptions in Business Logic**
  - Example: Inventory Update
  - Inside the CRUD layer:
    - Fetch product from database
    - If product does not exist:
      - Raise RecordNotFoundError
  - Validate stock:
    - Prevent negative values
    - Raise InsufficientStockError if violated

```python
def update_stock(self, db: Session, product_id: int, location_id: int,
                 quantity_change: int, reorder_point: Optional[int] = Non
    """...

    product = db.query(Product).filter(Product.id == product_id).first()
    if not product:
        raise RecordNotFoundError("Product", product_id)
    inventory_item = self.get(db, product_id, location_id)

    if not inventory_item:
        # Don't allow negative stock for non-existent items
        if quantity_change <= 0:
            return None
```

Additional Functionalities

- Automatically create inventory if missing
- Handle reorder levels

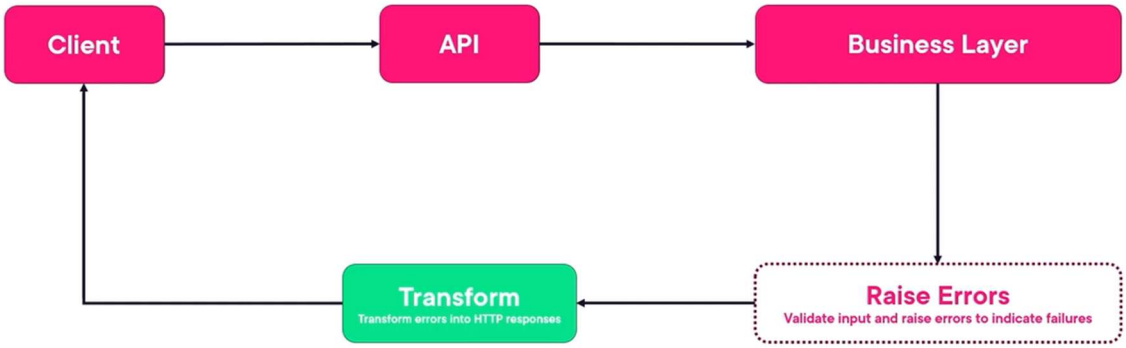
- Maintain data integrity

🚨 Global Exception Handling

Why Global Handlers?

Instead of handling errors in every route:

- Define centralized handlers
- Ensure consistent responses



Step-by-Step Implementation

- Create exception_handlers.py in core/
- Import custom exceptions

```
ore > error_handlers.py
from fastapi import Request
from fastapi.responses import JSONResponse

from app.core.exceptions import InventoryError
```

- Define handler functions

```python
async def inventory_exception_handler(request: Request, exc: InventoryError
    """Handler for our custom inventory exceptions"""
    return JSONResponse(
        status_code=exc.status_code,
        content={
            "error": True,
            "message": exc.message,
            "details": exc.details
        }
```

- Register handlers in main.py

```python
from app.core.error_handlers import inventory_exception_handler
```

Registering Handlers

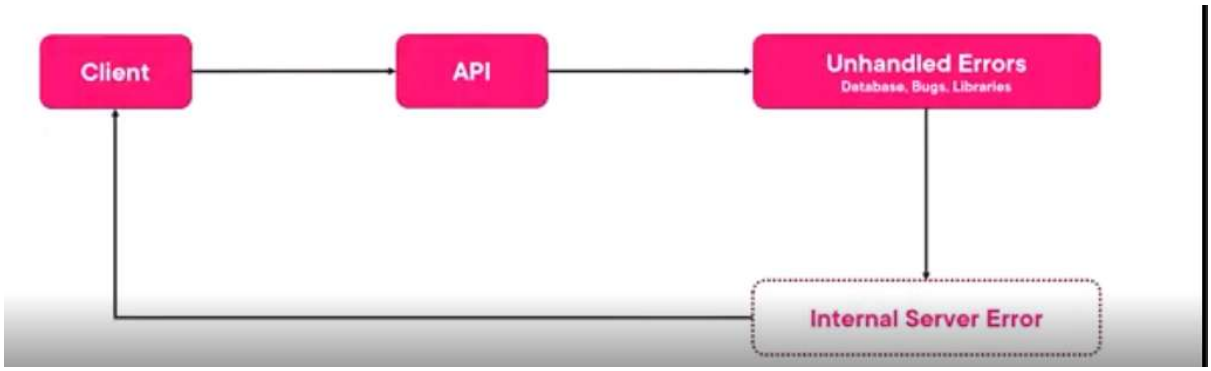
- Use:
  - app.add_exception_handler()

```python
app.add_exception_handler(InventoryError, inventory_exception_handler)
```

Handling Database Errors

Scenario

- Database connection fails
- Query execution error occurs



Step by step implementation

- Import SQLAlchemyError

```python
from sqlalchemy.exc import SQLAlchemyError
```

- Create handler function

```python
async def sqlalchemy_exception_handler(request: Request, exc: SQLAlchemyEr
    """Handler for SQLAlchemy database errors"""
    # Log the full exception details for debugging
    print(f"Database error: {str(exc)}")

    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={
            "error": True,
            "message": "A database error occurred",
```

```python
            "details": {
                "error_type": exc.__class__.__name__,
                # Provide a simplified message for the client
                "error": "The operation couldn't be completed due to a dat
            }
        }
    )
```

Behavior

- Returns:
  - Status: **500 Internal Server Error**
  - Message: Generic (to avoid leaking sensitive details)
  -
- Registration
  - Register handler in main.py
  - Map it to SQLAlchemyError
-

```python
from sqlalchemy.exc import SQLAlchemyError
```

```python
app.add_exception_handler(SQLAlchemyError, sqlalchemy_exception_handler)
```

General Exception Handler

Purpose

Catch any **unhandled exceptions** in the application.

Behavior

- Logs error for debugging

- Returns safe response to client

Advanced Scenario

If the database or table does not exist:

- System may:
  - Reinitialize resources
  - Or return a controlled failure

- Creating general exception handler to handle all type of error

```
async def general_exception_handler(request: Request, exc: Exception):
    """Handler for any unhandled exceptions"""
    # Log the full exception details for debugging
    print(f"Unexpected error: {str(exc)}")

    return JSONResponse(
        status_code=status.HTTP_500_INTERNAL_SERVER_ERROR,
        content={
            "error": True,
            "message": "An unexpected error occurred",
            "details": {
                "error_type": exc.__class__.__name
```

```
        "message": "An unexpected error occurred",
        "details": {                (property) __class__: type[Exception]
            "error_type": exc.__class__.__name__
            # Don't expose internal error details to clients
        }
    }
```

- Response Structure
  - error: true
  - message: user-friendly message
  - details: optional debugging info

- Register the general purpose exception handler in main.py file

```
app.add_exception_handler(Exception, general_exception_handler)
```

- Extending the Error Handling System:

  - In production systems, you must handle:

- Database errors
- Third-party service failures
- Network issues
- Unexpected runtime errors

3.4 Implementing structured responses and documentation

As APIs grow, it becomes critical not only to return data—but to return it in a **consistent, well-documented, and predictable format**.

Poorly structured APIs force developers to rely on:

- Trial and error
- Guessing field meanings
- Reverse engineering responses

This leads to fragile integrations and poor developer experience.

🧩 Why Structured Responses Matter

When APIs return raw database models:

- Internal structure leaks into external systems
- Changes in the database can break clients
- Documentation becomes unclear

Best Practice: Always define explicit response schemas using Pydantic.

Creating Response Models with Pydantic

Instead of returning SQLAlchemy models directly, we define **custom response models**.

Step-by-step implementation:

- Create Schema File

    - Navigate to schemas/
    - Create or update products.py
- **Define Response Model**
    - Create a class (e.g., ProductResponse)
    - Inherit from BaseModel
    - Define fields such as:
        - id
        - name
        - description
        - sku
        - price

```python
from pydantic import BaseModel,Field, field_validator
from typing import Optional
import re

class ProductCreate(BaseModel):...

class ProductResponse(BaseModel):
    id: int = Field(..., description="Unique product identifier")
    name: str = Field(..., description="Product name")
    description: Optional[str] = Field(None, description="Product descript
    sku: str = Field(..., description="Stock keeping unit code")
    price: float = Field(..., description="Product price in USD")

    model_config = {
        "from_attributes": True,  # Enable automatic conversion from SQLAl
        "json_schema_extra": {
            "example": {
                "id": 1,
                "name": "Wireless Keyboard",
                "description": "Ergonomic wireless keyboard with long batt
                "sku": "KB-1234-A1",
                "price": 49.99
```

Why This Approach?

- Decouples API from database layer
- Ensures consistent output
- Improves security (hide internal fields)
- Enables automatic documentation

Linking Response Model to Endpoints

In products.py (router):

- Use response_model in route decorator

What This Does

- Ensures responses follow defined schema
- Automatically validates outgoing data
- Generates structured API documentation

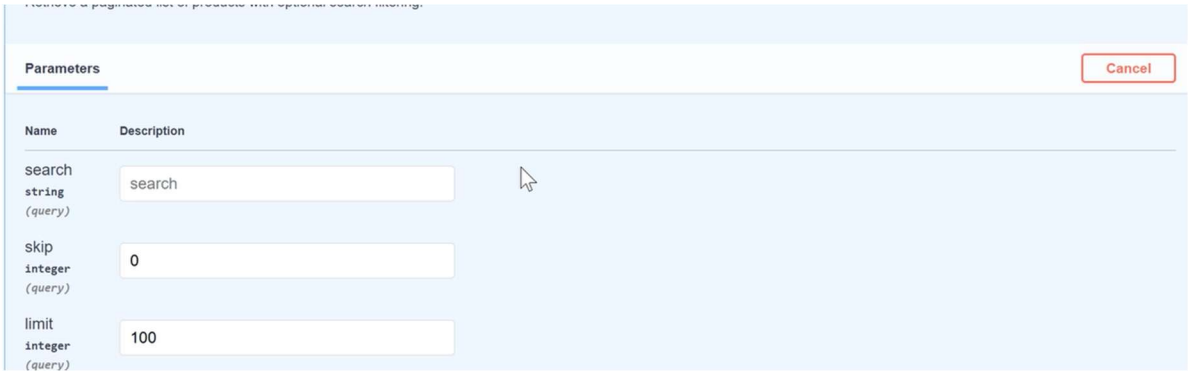
Example Behavior

- Endpoint: GET /products/
- Returns: List of ProductResponse objects

```
14 @router.get(
15     "/",
16     response_model=List[ProductResponse],,
17     summary="List all products",
18     description="Retrieve a paginated list of products with optional searc
19 )
```

Enhancing API Documentation

FastAPI auto-generates documentation, but we can enhance it further.



Adding Metadata

You can improve documentation by adding:

- summary → Short description
- description → Detailed explanation
- Parameter descriptions
- Constraints (min, max, default values)

Improving Query Parameters

Instead of plain parameters, add metadata:

- Description of purpose

- Default values
- Validation rules

Result

- Cleaner Swagger UI
- Better developer understanding
- Built-in validation

Advanced Query Parameter Design

You can extend parameters to support:

- Pagination (e.g., skip, limit)
- Search filters
- Sorting options

Each parameter can include metadata, making APIs self-explanatory.

Step by step :

- In products.py under router s file go to the api where we have all the parameters mentioned and you can augment the parameters by adding metadata associated with each field
- For adding metadata we need to import modules and libraries

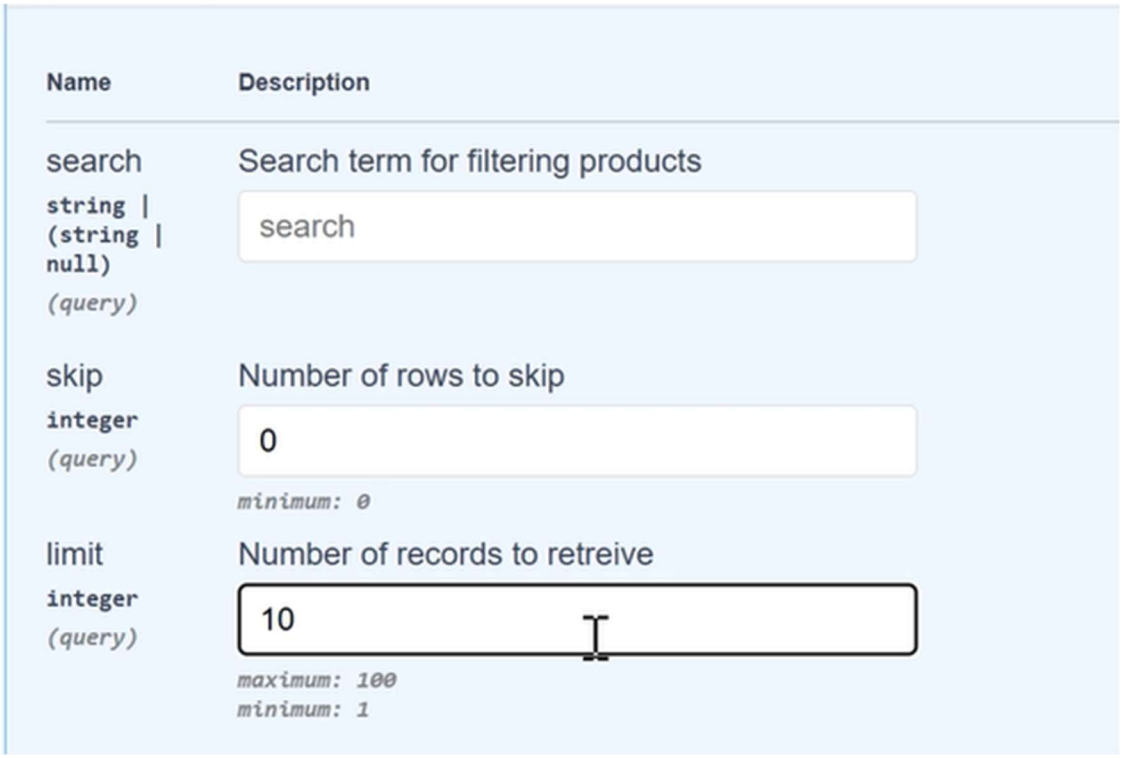
```
from typing import Optional,List
```

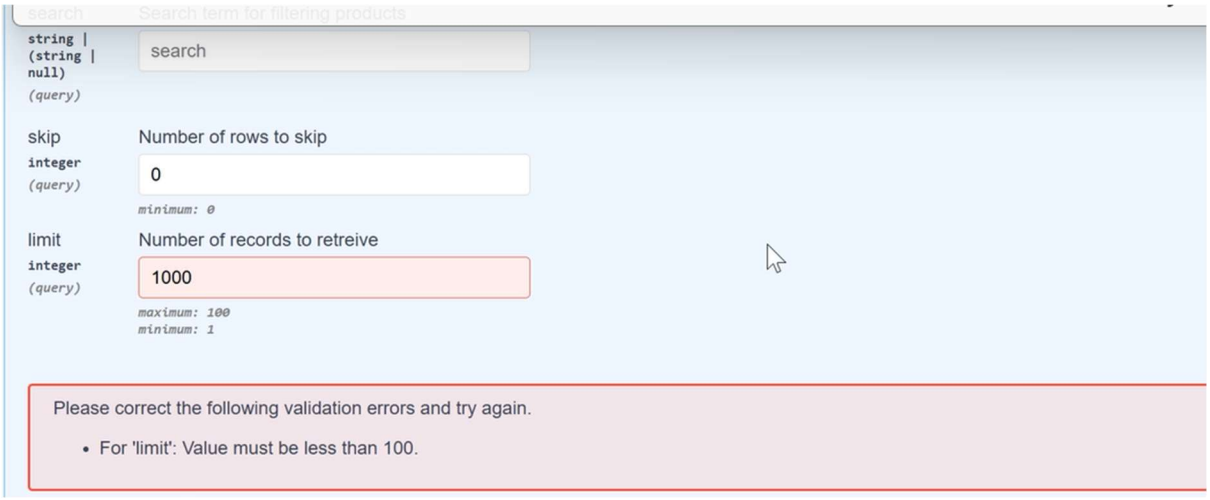
```
from fastapi import APIRouter, Depends,Query,status
```

- Making changes in the parameter of the function

```
@router.get(
    "/",
    response_model=List[ProductResponse],
    summary="List all products",
    description="Retrieve a paginated list of products with optional search filtering."
)
def list_products(
    search: Optional[str] = Query(None, description="Search term for filtering products"),
    skip: int = Query(0, ge=0, description="Number of rows to skip"),
    limit: int = Query(10, ge=1, le=100, description="Number of records to retreive"),
    db: Session = Depends(get_db)
):
```

- After adding metadata it looks way better





automatically it is validating the fields

Demo: Documenting API Responses

Documenting api endpoints returns for comprehensive response mapping

By default, APIs only show success responses. However, real systems must document **all possible responses**.

Example: Get Product by ID

Success Response

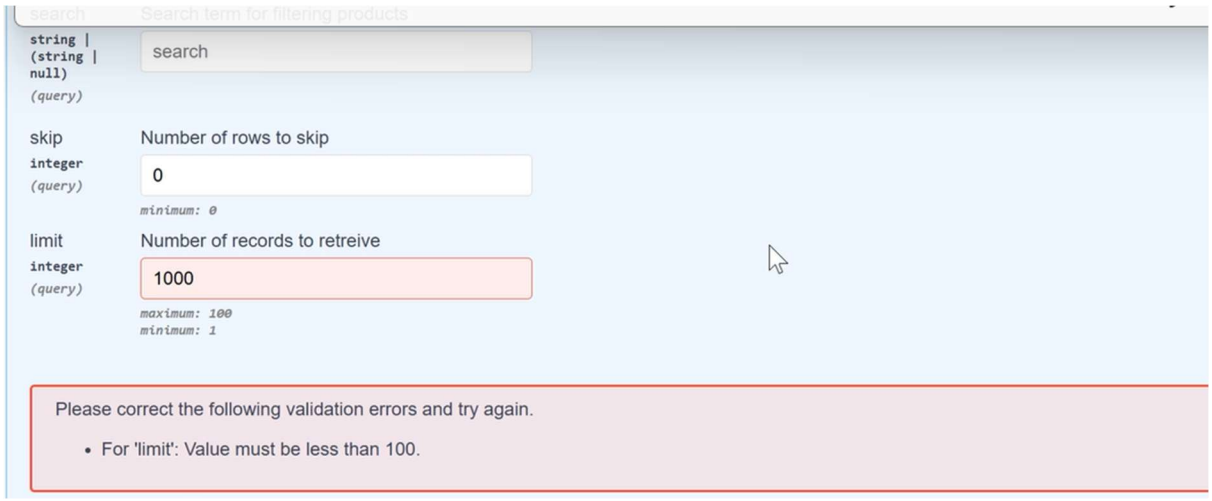
- Status: **200 OK**
- Returns: ProductResponse

Error Response

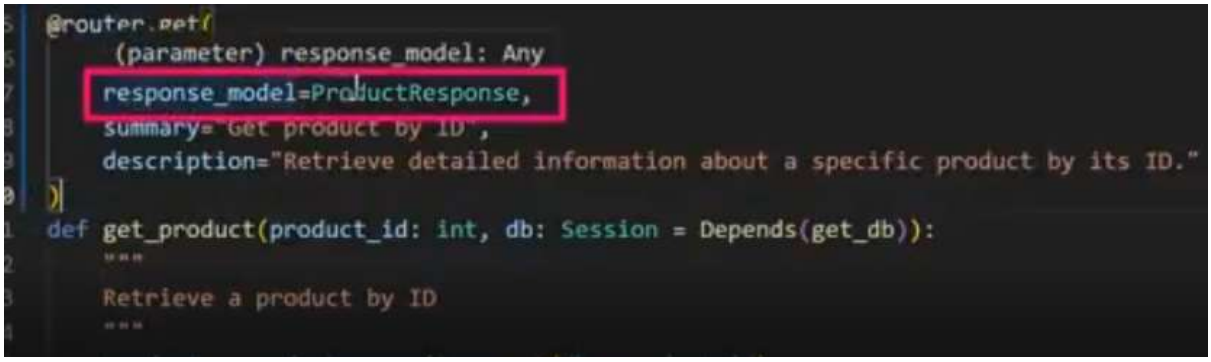
- Status: **404 Not Found**
- Occurs when product does not exist

Why Explicit Response Documentation Matters

- Developers know what to expect
- Reduces integration errors
- Improves API reliability



Need to change the all the path of epi endpoints to specify the product response



This is helps us adding an example schema

```
@router.get(
    "/{product_id}",
    response_model=ProductResponse,
    summary="Get product by ID",
    description="Retrieve detailed information about a specific product by its ID.",
    responses={
        status.HTTP_200_OK: {
            "description": "Product found successfully",
            "model": ProductResponse
        },
        status.HTTP_404_NOT_FOUND: {
            "description": "Product not found",
            "content": {
                "application/json": {
                    "example": {
                        "message": "Product not found",
                        "details": {
                            "resource_type": "Product",
                            "resource_id": "123"
                        }
```

But what happens when the product doesn't exist as 404 not found that is a common status codes that are raised when error happens but we can specify the responses

- **Endpoint:** The GET method is mapped to the URL path /{product_id}.
- **Success Response:** Returns HTTP 200 OK with detailed product information mapped to the ProductResponse model.
- **Error Handling:** Returns HTTP 404 Not Found if the product does not exist, including a specific JSON message body.

Defining Multiple Responses

You can explicitly define different responses in your endpoint:

- Success (200)
- Not Found (404)
- Validation Error (422)

Benefits

- Clear API contracts
- Better frontend-backend integration
- Improved debugging experience

Why This Helps

- Developers understand structure instantly

- Reduces guesswork
- Improves onboarding speed